Higher Order Functions

Defn.

- A function that takes another function as an argument

  or

- A function that returns a function as results

Why?

- Many functions share the same patterns
- Therefore can abstract these patterns into commonly used functions
- These can then be re-used
- Allows us to create new functions on top of existing ones, hence the name

3

3

Examples

- There are several higher order functions in Haskell
- They abstract common patterns of recursion
- Common ones include
  - map
  - filter
  - iterate
  - zipWith
  - foldl
  - foldr
  - scanr
  - scanl
  - takeWhile
  - dropWhile

4

4

map

- Signature

```
map::(a->b) ->[a] -> [b]
```

- Instances can include

```
(Integer -> Integer) -> [Integer] -> [Integer]
(Char -> Integer) ->[Char] -> [Integer]
```

- Takes a function and a list as arguments
- Applies the function to each element of the list
- Returns a new list

5

5

map

- Examples:

```
map (+3) [1,2,3] => [4,5,6]
map (3*) [1,2,3] => [3,6,9]
map (^2) [1,2,3] => [4,9,16]
map square [1..7] => ?
map (take 4) [“pumpkin”, “cow”, “mellow yellow”] =>?
map head [“dog”, “ostrich”, “pig”, ”elephant”] => ?
map reverse [“dog”, “cat”, “cow”] => ?
```

6

6

filter

- Signature

```
filter::(a -> Bool) -> [a] -> [a]
```

- Used to filter lists
- Takes a Boolean valued function (predicate) and a list as input
- Returns a sub-list such that each item in the new list satisfies the predicate
- That is, if a member of the list passes the test, it is added to the new list

7

7

filter

- Examples

```
filter even [1..5] => [2,4]
filter (>3) [1,5,3,4] => [5,4]
filter (==3) [1,5,3,4] => [3]
filter odd [3,6,7,9,12,14] => [3,7,9]
```

8

8

fold

- The fold functions are used to combine elements of a list in to one value
- The fold functions
  - take a function
  - a starting value called the *accumulator*
  - and a list to fold
- Depending on the direction of the fold, there are two functions: `foldr` and `foldl` for right and left folds respectively

9

9

foldl

- Signature

```
foldl::(a->b->a) -> a ->[b] ->a
```

- Folds a list from the left side
- Takes the accumulator and first element of the list
- Apply the function
- Feed the results to the function with the second element of the list, etc.

➢The function has the accumulator as the first parameter and the current list item as the second parameter

10

10

foldl

- Examples

```
foldl (+) 0 [1,2,3,4] => 10
foldl (+) 5 [1,2] => 8
foldl (/) 64 [4,2,4] => ?
foldl max 5 [1,2,3,4] => ?
foldl max 5 [1,2,3,4,5,6,7] => ?
foldl (-) (-5) [1,2] => ?
```

11

11

foldr

- Signature

```
foldr::(a->b>b) -> b -> [a] -> b
```

- Folds the list from the right side
- The foldr function
  - takes the last element of the list and the accumulator
  - apply the function
  - then take the next element from the end of the list

➢ The function takes the current element as the first parameter and the accumulator as the second parameter

12

12

foldr

- Examples

```
foldr (+) 0 [1,2,3,4] => 10
foldr (+) 5 [1,2] => 8
foldr (-) (-5) [1,2] =>
foldr (-) 0 [1,2] =>
foldr (*) 1 [1,2] =>
foldr (*) 3 [3,4] =>
Foldr (/) 2 [8,12,24,4] =>
```

13

13

foldr & foldl

- Main difference between the right and left folds is that *foldr* works on infinite lists whilst *foldl* doesn't
- When you take an infinite list and fold from the right, you will eventually reach the beginning of the list
- Folding from the left, you will never reach the end of the list

14

14

dropWhile

- Signature

```
dropWhile::(a -> Bool) -> [a] -> [a]
```

- Creates a list from another list
- Inspect the original list using the Boolean function
- Takes elements from where the condition fails until the end of the list

15

15

dropWhile

- Examples

```
dropWhile (<3) [1,2,3,4,5] => [3,4,5]
dropWhile even [2,4,6,7,9,11,12,13,14] => [7,9,11,12,13,14]
dropWhile (< 3) [1,2,3,4,5,1,2,3] => ?
dropWhile (< 9) [1,2,3] => ?
dropWhile (< 0) [1,2,3] =>?
```

16

16

takeWhile

- Signature

```
takeWhile::(a->Bool) -> [a] ->[a]
```

- Also creates a list from another list
- Inspects the original list
- Takes elements from this list until the condition fails, then stop

➢Returns the longest prefix (possibly empty) of the list of elements that satisfy the predicate.

17

17

takeWhile

- Examples

```
takeWhile (< 3) [1,2,3,4,5] => [1,2]
takeWhile (>3) [1,2,3,4,5] => []
takeWhile odd [1,3,5,7,9,10,11,13,15] => [1,3,5,7,9]
takeWhile ('W'>) "hello world" => "hello"
takeWhile (< 9) [1,2,3] => ?
takeWhile (< 0) [1,2,3] => ?
```

18

18

zipWith

- Signature

```
zipWith::(a -> b -> c) -> [a] -> [b] -> [c]
zipWith f [x1,x2,x3..] [y1,y2,y3..] == [f x1 y1, f x2 y2, f x3 y3..]
```

- Creates a new list by combining two input lists using a function
- The elements of the new list are calculated using the function and the elements at the same positions in the input lists

19

19

zipWith

- Examples

```
zipWith (+) [1,2,3]  [3,2,1] => [4,4,4]
zipWith (+) [1, 2, 3] [4, 5, 6] => [5,7,9]
zipWith (*) [1,2,3] [3,2,2] => ?
zipWith (*) [3,5,6] [3,4,5] => ?
zipWith (**)[1..10] [1..10] => ?
```

20

20

iterate

- Signature

```
iterate::(a -> a) -> a -> [a]
```

- *iterate* takes a function and starting value
- Then it applies the function to the starting value, then it applies the same function to the result, and so on.
- This creates an infinite list

➢The first element of the new list is the starting value

➢The next element is calculated by applying the function on the starting value, the next element by applying the function on the previous result etc.

21

21

iterate

- Examples

```
iterate (2*) 1 => [1,2,4,8,16,32,64,128,256,512 …]
iterate (+3) 42 => [42,45,48,51,54,57,60,63,66,69 …]
iterate (^2) 2 => [2,4,16,256,65536…]
iterate ((+3)*2) 1 => ?
iterate not True => ?
```

22

22

scanl

- Signature

```
scanl :: (b -> a -> b) -> b -> [a] -> [b]
```

- scanl is similar to foldl, but returns a list of successive reduced values from the left

23

23

scanl

- Examples

```
scanl (+) 0 [1..4] => [0,1,3,6,10]
scanl (+) 42 [] => [42]
scanl (-) 100 [1..4] => [100,99,97,94,90]
scanl (+) 0 [1..] => [0,1,3,6,10,15,21,28,36,45]
```

24

24

scanr

- Signature

```
scanr :: (a -> b -> b) -> b -> [a] -> [b]
```

- scanr is the right-to-left dual of scanl
- Note that the order of parameters on the accumulating function are reversed compared to scanl

25

25

- Examples

```
scanr (+) 0 [1..4] => [10,9,7,4,0]
scanr (+) 42 [] => [42]
scanr (-) 100 [1..4] => [98,-97,99,-96,100]
```

26

26